



AULA 1

Problemas de classificação

A PROFESSORA

Tatiana Escovedo

Professora do Departamento de Informática da PUC-Rio, onde coordena cursos de pós-graduação lato sensu e colabora com pesquisas nas áreas de ciência de dados e engenharia de software. Atua como gerente geral de Transformação Digital da Petrobras. É doutora em engenharia elétrica, na área de métodos de apoio à decisão, e mestre em informática, na área de engenharia de *software*, pela PUC-Rio, e autora dos livros Introdução a *Data Science* – algoritmos de *Machine Learning* e métodos de análise; Jornada Java; Jornada *Python* e Engenharia de *Software* para Ciência de Dados - Um guia de boas práticas com ênfase na construção de sistemas de *Machine Learning* em *Python*.

O CONVIDADO

Adriano Koshiyama

Adriano Koshiyama é fundador da Holistic AI, uma start-up focada em fornecer uma solução de plataforma como serviço para organizações que desejam aproveitar a IA com ética e segurança. Além disso, ele é pesquisador em ciência da computação na University College London e, academicamente, publicou mais de 50 artigos em conferências e periódicos internacionais. É responsável por resultados inovadores na interseção de Machine Learning e Finanças, com o trabalho anterior em GANs e Transfer Learning nesta área.



Objetivos de aprendizagem da aula

Ao final desta aula, você irá:

- Compreender como funciona o aprendizado supervisionado em *machine learning*.
- Reconhecer problemas de classificação no contexto de *machine learning*.
- Diferenciar as métricas de avaliação de modelos de classificação.
- Entender o funcionamento dos principais algoritmos de classificação.
- Construir um modelo de classificação.

Que história é essa?

O objetivo desta disciplina é introduzir os principais conceitos de problemas de classificação e regressão, bem como de aprendizado supervisionado de *machine learning*, e implementá-los com exemplos na linguagem Python, que será utilizada durante todo o curso.

Ao longo desta disciplina, você terá aulas sobre esses temas e acesso a materiais preparados exclusivamente para que sua aprendizagem seja leve e dinâmica.

Nesta primeira aula, examinaremos o funcionamento do aprendizado supervisionado em *machine learning* e aprenderemos a reconhecer as principais características dos problemas de classificação. Além disso, veremos as principais métricas de avaliação utilizadas para modelos de classificação, entendendo como funcionam os algoritmos de *machine learning* utilizados em problemas de classificação. Por fim, veremos como construir esses modelos na prática.

Aprendizado supervisionado

No contexto de ciência de dados, muitas vezes trabalhamos com modelos de *machine learning* (aprendizado de máquina), para construirmos modelos estatísticos que “aprendam” com nossos dados e sejam capazes de fazer predições.

Esse aprendizado pode ser categorizado em quatro tipos:

Supervisionado



Quando o modelo de conhecimento é construído a partir dos dados apresentados na forma de pares ordenados, como entrada e saída desejada.

Não supervisionado



Quando não existe a informação da saída desejada e o processo de aprendizado busca identificar regularidades entre

os dados, a fim de agrupá-los em função das similaridades que apresentam entre si.

Semissupervisionado



Combina o aprendizado supervisionado e o não supervisionado.

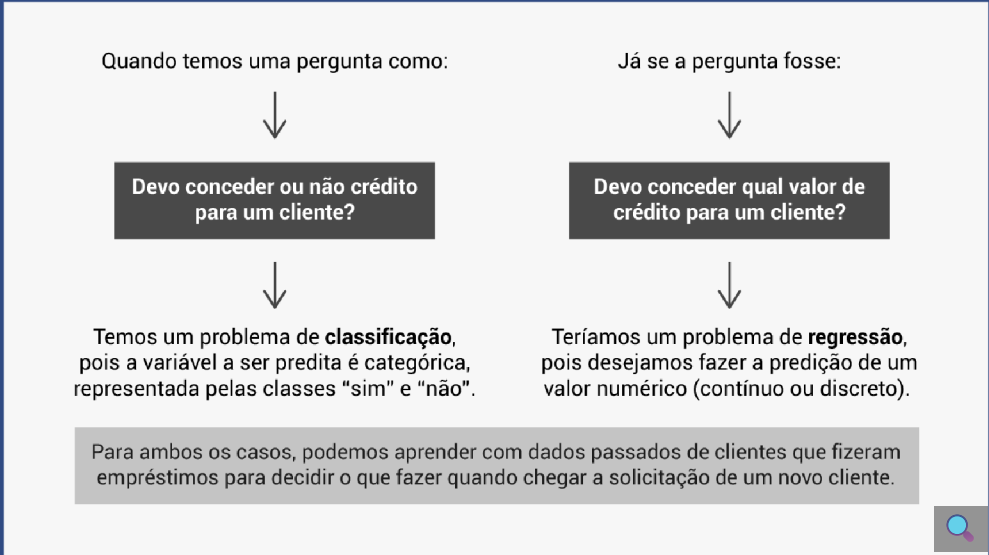
Por reforço



Quando a máquina é capaz de “perceber” o estado do ambiente, executar ações de acordo com ele, receber “recompensas” pelas ações executadas e trocar de estado, se apropriado.

Nesta disciplina, vamos focar o aprendizado supervisionado, que trata dos problemas de **classificação** e **regressão**.

Acompanhe na imagem:



No **aprendizado supervisionado**, o modelo é construído a partir dos dados de entrada (também chamados de *dataset*), que são apresentados para um algoritmo na forma de pares ordenados (entrada-saída desejada).

Dizemos que esses dados são **rotulados**, pois sabemos de antemão a saída esperada para cada entrada.

Nesse caso, o aprendizado (ou treinamento) consiste em apresentarmos para o algoritmo um número suficiente de exemplos (também chamados de registros ou instâncias) de entradas e saídas desejadas (já rotuladas previamente). Assim, o objetivo do algoritmo é **aprender uma regra geral** que mapeie as entradas nas saídas corretamente, consistindo no modelo final.



Os dados de entrada podem ser divididos em dois grupos:

Interativo

Descrição do interativo

É comum que particionemos os dados de entrada (rotulados) em dois conjuntos:

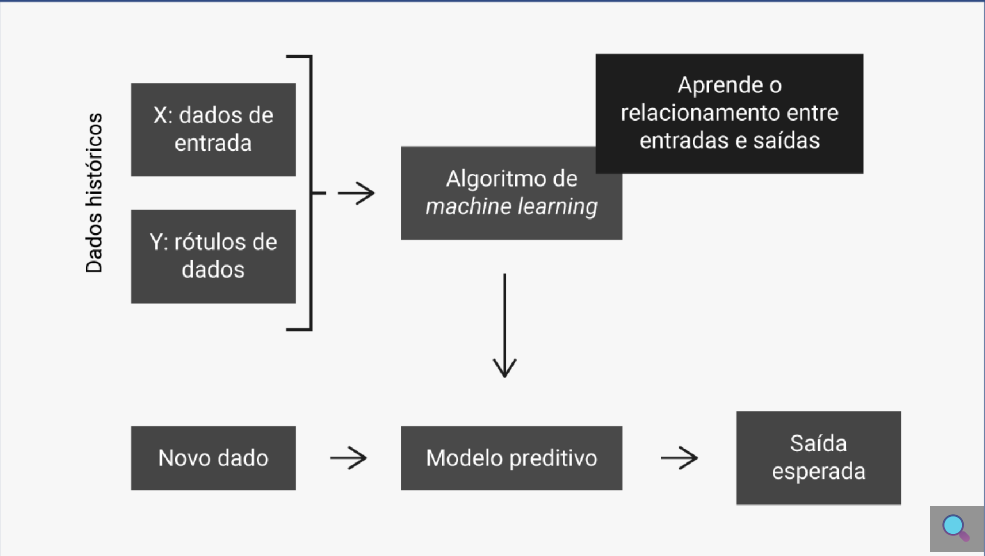
De treinamento

Servirá para construir o modelo.

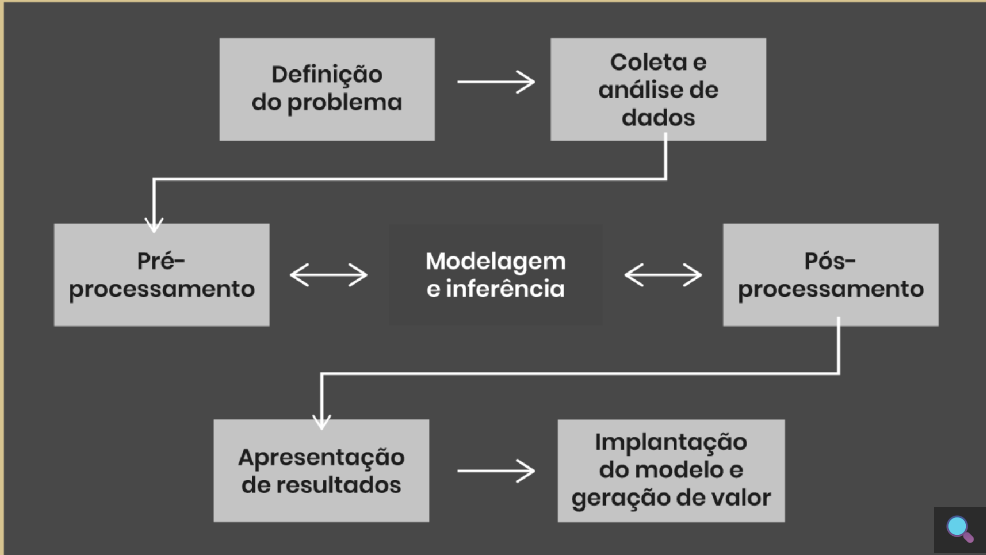
De teste

Também chamado na literatura de conjunto de validação, servirá para verificar como o modelo se comportaria em dados não vistos, de forma que possamos ajustá-lo, se necessário, para a construção final do modelo a ser aplicado a novos dados em que ainda não conhecemos a saída esperada.

Acompanhe o esquema do aprendizado supervisionado:



Já a figura a seguir ilustra o esquema básico de um projeto de ciência de dados, que pode ser resumido em sete etapas:



Dentro do ciclo de vida de projetos de ciência de dados, a etapa de **modelagem e inferência** – destacada na figura – consiste, de forma simplificada, em escolher o modelo mais adequado para resolver o problema em questão.

Resumidamente, essa etapa envolve três tarefas:

Interativo

Descrição do interativo

Em problemas de aprendizado supervisionado, na etapa de **modelagem**, apresentamos para o algoritmo um número suficiente de exemplos (também chamados de registros ou instâncias) de entradas e saídas desejadas, já rotuladas previamente – ou seja, **treinamos o modelo**.

Nessa etapa de treinamento, o objetivo do algoritmo é aprender uma regra geral que mapeie as entradas nas saídas corretamente.



Fique ligado

- Em problemas de classificação, a saída é sempre categórica (classe).
- Em problemas de regressão, a saída é sempre numérica (discreta ou contínua).
- Para ambos os problemas, dado um novo padrão, espera-se que o modelo estime a classe ou o valor mais esperado para a variável resposta.

A seguir, conheça mais a fundo os problemas de classificação.

Problemas de classificação

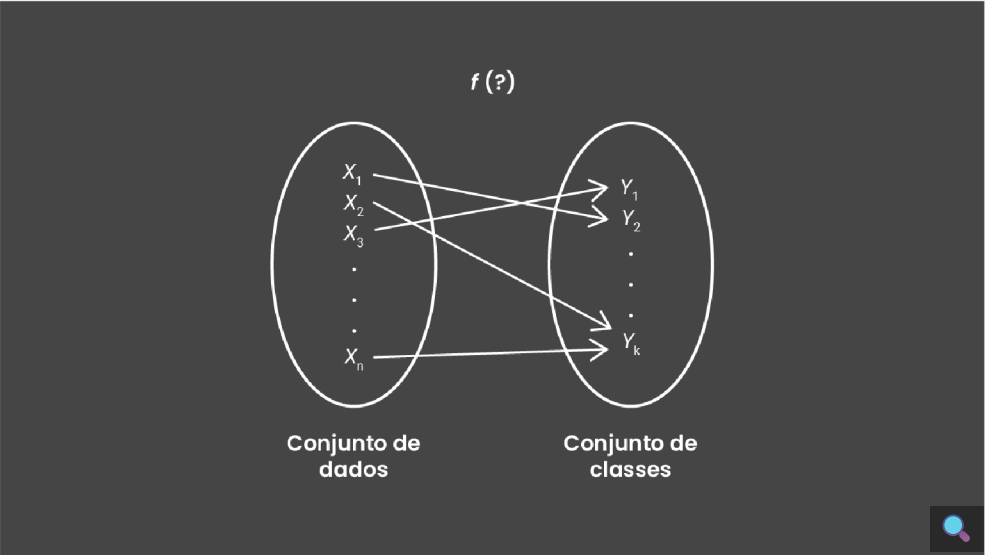
A classificação é uma das tarefas de *machine learning* mais importantes e populares. Conforme já mencionado, seus problemas utilizam o aprendizado supervisionado.



Podemos definir um problema de classificação, informalmente, como a busca por uma função matemática que permita associar corretamente cada exemplo X_i de um conjunto de dados a um único rótulo categórico (Y_i), denominado *classe*.

Vimos, também, que os dados de entrada podem ser divididos em dois grupos: X , com os atributos a serem utilizados na determinação da classe de saída; e Y , com o atributo para o qual se deseja fazer a predição do valor da classe.

Uma vez identificada, essa função pode ser aplicada a novos exemplos para prever as classes em que se enquadram. A figura a seguir ilustra esse problema:



Formalmente, podemos definir um problema de classificação como: dada uma coleção de exemplos f , obter uma função (hipótese) h que seja uma aproximação de f .

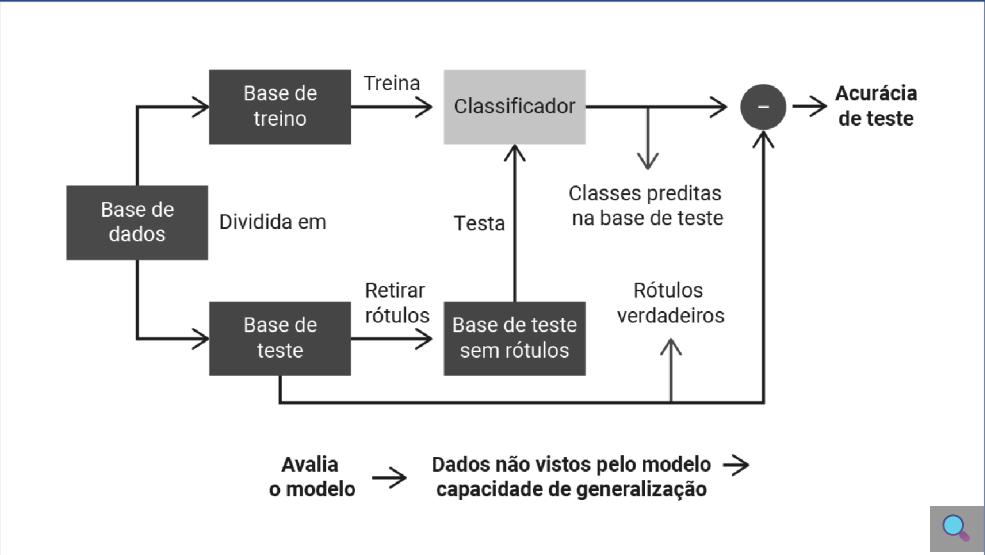
A imagem de f é formada por rótulos de classes retirados de um conjunto finito, e toda hipótese h é chamada de **classificador**. O aprendizado consiste, então, na busca da hipótese h que mais se aproxime da função original f .

O fluxo de um problema de classificação pode ser representado da seguinte forma:

Interativo

Descrição do interativo

Esse fluxo pode ser resumido por esta figura:



Quando construímos um modelo de aprendizado supervisionado, estamos interessados na **generalização**, etapa em que aplicamos um modelo aos dados que não foram utilizados para construí-lo – ou seja, novos dados.



Podemos pensar que cada *dataset* é uma amostra finita de uma população e que os dados de treinamento também vieram dessa população. Queremos que os modelos sejam bons não apenas para o conjunto de treinamento usado para construir o modelo, mas também para a população em geral.

Para isso, podemos usar a técnica de *holdout* (separação do *dataset* em bases de treino e teste): o modelo é construído com um conjunto de dados e seu erro de generalização é avaliado com outro, não utilizado para o treinamento.

Vale a pena observar que, uma vez identificada uma hipótese (treinado um classificador), esta pode ser muito específica para o conjunto de treino utilizado. Caso esse conjunto não corresponda a

uma amostra suficientemente representativa da população, esse classificador **pode ter um bom desempenho no conjunto de treino, mas não no conjunto de teste.**



Fique ligado

Dizemos que ocorreu *overfitting* quando o classificador se ajustou em excesso ao conjunto de treinamento.

Geralmente, o erro de teste é maior do que o erro de treinamento. Idealmente, esses erros são próximos, e o modelo tem boa capacidade de generalização, sendo capaz de aprender exemplos ainda não vistos no treinamento.



Se o erro de teste é grande demais em relação ao de treino, isso é um indicativo de que pode estar ocorrendo *overfitting* e de que o **modelo está memorizando os padrões de treinamento** em vez de descobrir regras ou padrões generalizados. Em geral, modelos mais simples tendem a generalizar melhor.

Um modelo bem ajustado apresentará erros de mesma magnitude tanto nos novos dados quanto nos dados de treino, enquanto um modelo com *overfitting* “memoriza” os dados de treino e apresenta erros muito maiores nos novos dados.

Outra situação que pode acontecer é quando o algoritmo de aprendizado tem parametrizações inadequadas.



Mantendo o foco

Nesse caso, diz-se que ocorreu *underfitting*: o classificador se ajustou pouco ao conjunto de treinamento, não sendo adequado para realizar previsões no conjunto de teste.

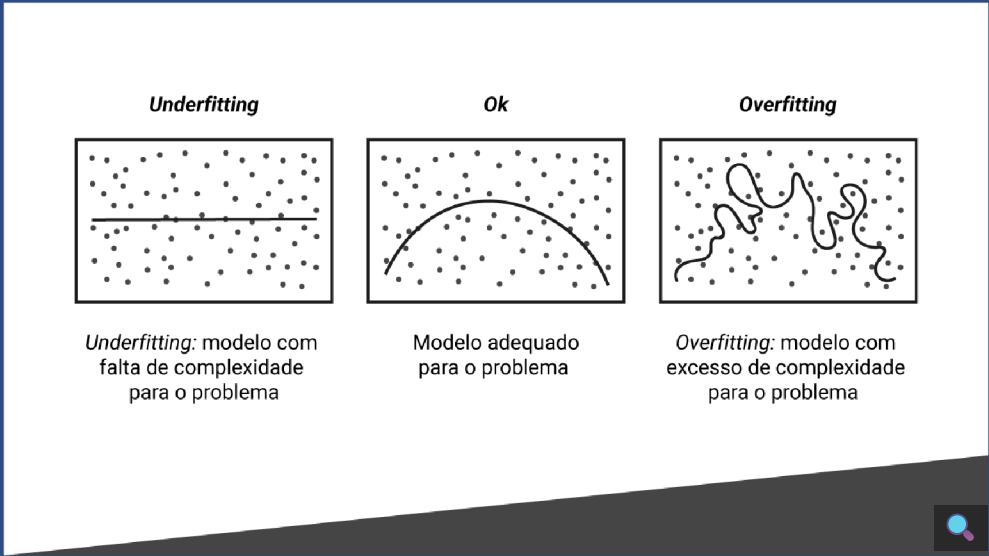
Para resolver esse problema, podemos experimentar um modelo mais complexo e que seja capaz de aprender os padrões do conjunto de treinamento de forma mais adequada.



Na aprendizagem supervisionada, ao mesmo tempo em que o modelo preditivo precisa ser suficientemente flexível para aproximar os dados de treinamento, o processo de treinamento deve evitar que o modelo absorva os ruídos da base. O modelo deve capturar as regularidades dos dados de treinamento, mas também generalizar bem para dados desconhecidos (conjunto de teste).

A solução para esse problema é escolher o momento adequado para interromper o treinamento e/ou utilizar validação cruzada, como veremos nos exemplos práticos desta disciplina.

A figura a seguir ilustra os casos de modelos de classificação com *underfitting* e *overfitting*, bem como o modelo adequado para o problema:



Chegamos, então, a um dilema:

- Por um lado, queremos que nosso algoritmo modele muito bem os dados de treinamento, caso contrário, perderemos recursos relevantes e tendências interessantes.
- Por outro lado, não queremos que nosso modelo se encaixe muito perfeitamente aos dados de treinamento, arriscando aprender erros e irregularidades.



Buscando evitar o *overfitting*, podemos ir para o outro extremo, fazendo com que nosso modelo ignore recursos importantes do *dataset*. Isso acontece quando escolhemos um modelo que não é complexo o suficiente para capturar esses recursos importantes, ocorrendo *underfitting*.

Na aprendizagem supervisionada, ao mesmo tempo em que o modelo preditivo precisa ser suficientemente flexível para aproximar os dados de treinamento, o processo de treinamento deve evitar que o modelo absorva os ruídos da base.

O modelo deve capturar as regularidades dos dados de treinamento, mas também generalizar bem para dados desconhecidos. Isso é conhecido como **dilema bias x variância**, e fala sobre a importância do equilíbrio entre *underfitting* e *overfitting*.

Como mencionado anteriormente, uma das formas de lidar com o dilema *bias* x variância é utilizar a **validação cruzada** (*cross validation*), que estima o desempenho de generalização do modelo não apenas uma vez (como no *holdout*), mas diversas vezes (*k*), fornecendo estatísticas sobre o desempenho estimado (como média e variância) para que possamos entender como é esperado que o desempenho varie entre os conjuntos de dados, tendo mais confiança na estimativa de desempenho.

Interativo

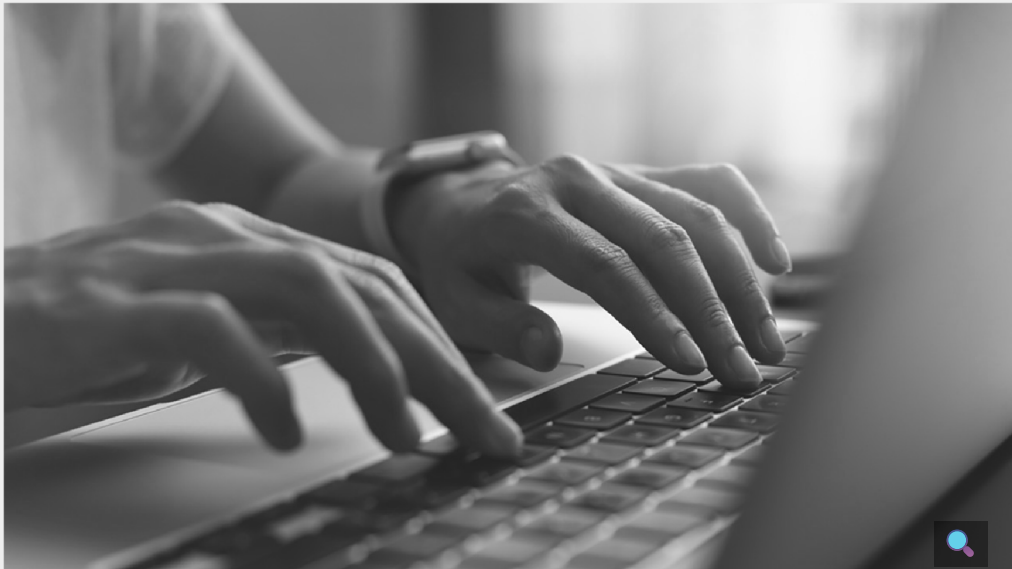
Descrição do interativo

Assim, a validação cruzada faz um melhor uso de um conjunto limitado de dados, utilizando todos os dados tanto para treino (*k*-1 vezes) como para teste (1 vez). Cada repetição produz um modelo e uma estimativa de generalização, e calculamos a média e o desvio-padrão das estimativas de todos os *k folds* de teste. Essa estimativa geralmente é mais realista do que quando utilizamos o *holdout*, que usa apenas o conjunto de teste para isso.

Você vai aprender mais sobre as métricas de avaliação a seguir.

Métricas de avaliação

Vale a pena mencionar o teorema “não existe almoço grátis”, ou seja, não existe um algoritmo de aprendizado que seja superior aos demais, quando considerados todos os problemas possíveis.



A cada problema, os algoritmos disponíveis devem ser experimentados, com diversas configurações, para identificar os que obtêm melhor desempenho. Para isso, precisamos utilizar **métricas de avaliação** apropriadas. Existem diversas medidas para estimar o desempenho de um modelo de aprendizado supervisionado, ou seja, para avaliar o modelo.

Para problemas de classificação, a **matriz de confusão** oferece um detalhamento do desempenho do modelo, mostrando, para cada classe, o número de classificações corretas em relação ao número de classificações indicadas pelo modelo, como ilustra a figura:

Classes	Predita C1	Predita C2
Verdadeira C1	Verdadeiros positivos	Falsos negativos
Verdadeira C2	Falsos positivos	Verdadeiros negativos

Para problemas de classificação, a métrica mais utilizada é a **acurácia**, ou taxa de acerto do classificador, dada por:

$$Acc(h) = 1 - Err(h)$$

A acurácia é uma função da taxa de erro (ou taxa de classificação incorreta), dada por:

$$Err(h) = \frac{1}{n} \sum_{i=1}^n ||y_i \neq h(i)||$$

Em que:

- o operador $||E||$ retorna 1, se a expressão E for verdadeira, e 0, caso contrário;
- n é o número de exemplos (registros da base de dados);
- y_i é a classe real associada ao i -ésimo exemplo;

- $h(i)$ é a classe indicada pelo classificador para o i -ésimo exemplo.

Essa é uma métrica útil quando os erros nas predições de todas as classes são igualmente importantes. A acurácia também pode ser extraída da matriz de confusão, pois representa o número de exemplos classificados corretamente, dividido pelo número total de exemplos classificados:

$$Acurácia = \frac{VP+VN}{VP+VN+FP+FN}$$

Também a partir da matriz de confusão, temos a precisão e o *recall*:

Precisão

É a razão entre o número de predições VP e a quantidade total de predições positivas:

$$Precisão = \frac{VP}{VP + FP}$$

Recall

É a razão entre o número de predições VP e a quantidade total de exemplos positivos no *dataset*:

$$Recall = \frac{VP}{VP + FN}$$

Na prática, quase sempre temos que escolher entre uma alta precisão ou um alto *recall*. Tipicamente, é impossível ter ambos.

Por fim, a **curva ROC** (*receiver operating characteristic*) contrasta os benefícios de uma classificação correta (TVP, sensibilidade ou *recall*) e o custo de uma classificação incorreta (TFP ou 1-especificidade):

Sensibilidade

Capacidade de identificar corretamente os indivíduos que apresentam a característica de interesse. É similar ao *recall*, que vimos anteriormente.

$$Sensibilidade = TVP = \frac{VP}{VP + FN}$$

Especificidade

Capacidade de identificar corretamente os indivíduos que NÃO apresentam a característica de interesse.

$$Especificidade = \frac{VN}{FP + VN}$$

$$1 - Especificidade = TFP = \frac{FP}{FP + VN}$$

Utilizamos a curva ROC para calcular a métrica **AUC** (*area under curve*), que varia entre 0 (predições 100% incorretas) e 1 (predições 100% corretas):

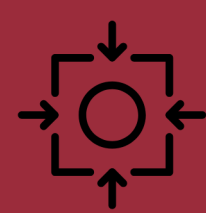


Algoritmos de *machine learning* para classificação

A seguir, serão apresentados alguns algoritmos de *machine learning* que podem ser usados para problemas de classificação.

KNN (*k-nearest neighbours*)

O algoritmo KNN (*k-nearest neighbours*, ou *k-vizinhos* mais próximos) é simples de entender e funciona muito bem na prática. É um algoritmo não paramétrico que não assume premissas sobre a distribuição dos dados.

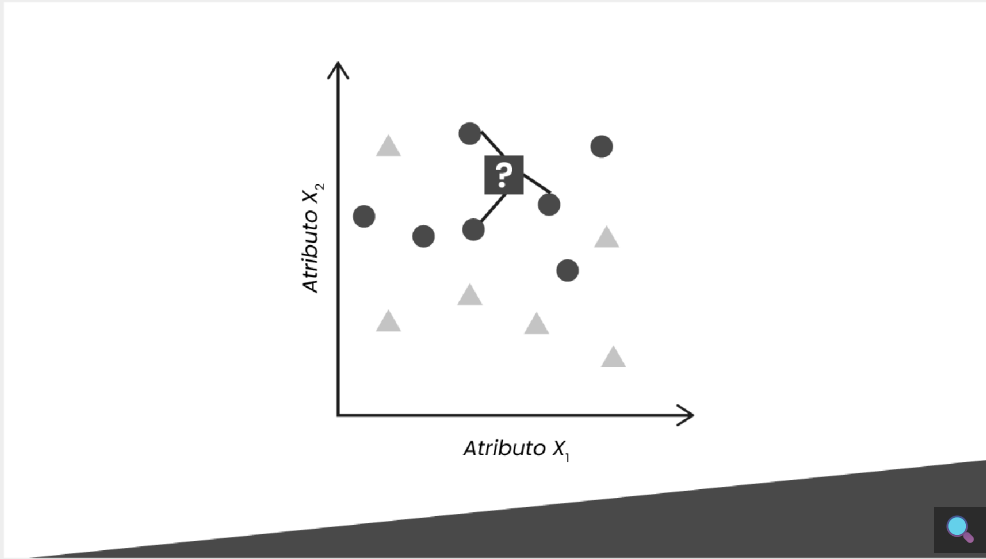


Mantendo o foco

Sua ideia principal é considerar que os exemplos vizinhos são similares ao exemplo cuja informação se deseja inferir.

Assim, o KNN utiliza uma métrica de distância para encontrar as *k* *instâncias* mais semelhantes nos dados de treinamento para uma nova instância. Além disso, considera a classe mais comum entre os vizinhos como a predição da classe da nova instância.

A figura a seguir ilustra essa ideia para problemas de classificação: o KNN considera que os registros do conjunto de dados correspondem a pontos no R^n , em que cada atributo corresponde a uma dimensão desse espaço:



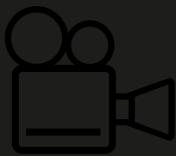
No KNN, inicialmente, o conjunto de dados (rotulado) é armazenado. Quando um novo registro deve ser classificado, é comparado a todos os registros do conjunto de treinamento para identificar os k (parâmetro de entrada) vizinhos mais próximos (mais semelhantes), de acordo com alguma métrica de distância (por exemplo, distância euclidiana).

A classe do novo registro é determinada por inspeção das classes desses vizinhos mais próximos, de acordo com a métrica escolhida. Na maioria das implementações do KNN, os atributos são normalizados, para que tenham a mesma contribuição na predição da classe ou do valor.

As etapas a seguir resumem o algoritmo KNN:

- 1 Definição da métrica de distância e valor de k .
- 2 Cálculo da distância do novo registro a cada um dos registros existentes no conjunto de referência.
- 3 Identificação dos k registros do conjunto de referência que apresentaram menor distância em relação ao novo registro (mais similares).
- 4 Apuração da classe mais frequente entre os k registros identificados no passo anterior usando votação majoritária.

Apesar de ser um algoritmo muito utilizado, o KNN tem algumas limitações: a performance de predição pode ser lenta em *datasets* grandes; é sensível a características irrelevantes, uma vez que todas as características contribuem para o cálculo da distância e, consequentemente, para a predição; e é necessário testar diferentes valores de k e a métrica de distância a utilizar.



Exemplo de KNN

Assista ao vídeo a seguir para ver um exemplo de funcionamento do KNN e entender melhor esse algoritmo.



A seguir, saiba mais sobre as Árvores de Classificação.

Árvore de Classificação

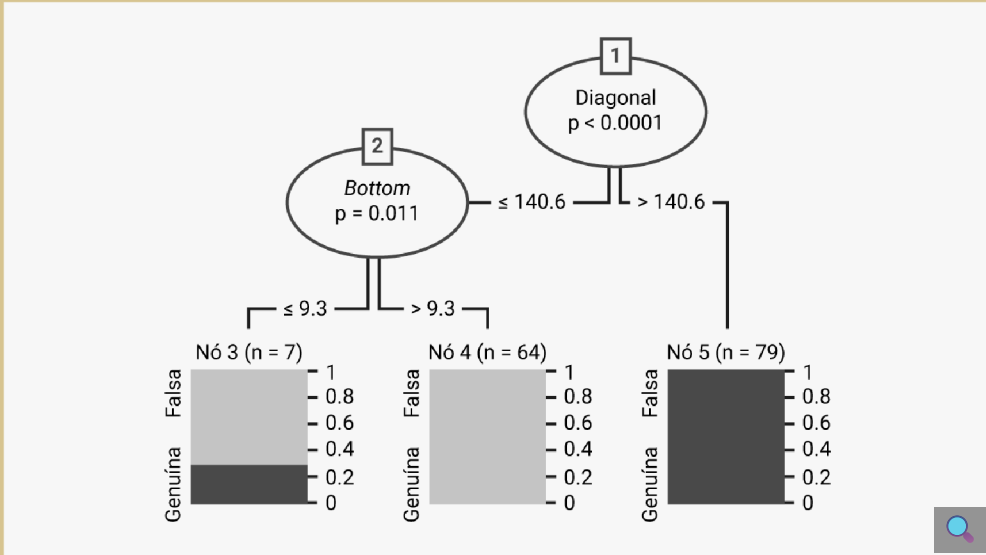
A Árvore de Decisão é um dos modelos preditivos mais simples de ser interpretado, e é inspirada na forma como humanos tomam decisões. Uma de suas principais vantagens é apresentar a informação visualmente, de uma forma fácil de entender.



As árvores podem ser usadas para problemas de classificação (nesse caso, são chamadas de Árvores de Classificação) ou regressão (Árvores de Regressão). Basicamente, uma Árvore de Decisão usa **amostras das características dos dados para criar regras de decisão** no formato de árvore, isto é, reduz os dados em um conjunto de regras que podem ser usadas para uma decisão.

As Árvores de Decisão aliam **acurácia e interpretabilidade**. Elas possibilitam a seleção automática de variáveis para compor suas estruturas: cada nó interno representa uma decisão sobre um atributo que determina como os dados estão particionados por seus nós-filhos.

Para fazer a predição de um novo exemplo, basta testar os valores dos atributos na árvore e percorrê-la até atingir um nó-folha (resultado predito). A figura a seguir ilustra uma Árvore de Classificação:



Para construir uma Árvore de Classificação a partir do nó-raiz, os dados são particionados usando uma estratégia de divisão e conquista, de acordo com a característica que levará ao resultado mais homogêneo após a separação ser realizada. Essa homogeneidade é medida por entropia ou medidas similares.

Há diferentes algoritmos para a elaboração de uma Árvore de Decisão, como:

- ID3
- CTree;
- C4.5;
- C5.0;
- CART.



Fique ligado

Todos os algoritmos são parecidos: em geral, a construção da árvore é realizada de acordo com alguma abordagem recursiva de particionamento do conjunto de dados, e a principal distinção está nos processos de **seleção de variáveis, critério de particionamento e critério de parada para o crescimento da árvore.**

Embora sejam intuitivas e interpretáveis, as Árvores de Decisão não costumam ter o mesmo nível de precisão preditiva que alguns outros algoritmos populares de aprendizado de máquina.



Elas tendem a criar limites de decisão excessivamente complicados, resultando em maior variação do modelo, levando a *overfitting*. Para evitar esse problema, podemos usar **algoritmos de poda (*pruning*)** para reduzir a profundidade e a complexidade da árvore, removendo nós

maiores que uma certa profundidade, para que a Árvore de Decisão não se ajuste em excesso aos dados de treinamento.

Entre os algoritmos de Árvores de Classificação existentes, um dos mais conhecidos é o C4.5, que utiliza conceitos e medidas de teoria da informação, uma disciplina que estuda a quantificação, o armazenamento e a comunicação da informação.

Interativo

Descrição do interativo

As etapas a seguir resumem o algoritmo C4.5 para problemas de classificação:

1. Calcular a entropia do conjunto T completo.
2. Para cada atributo, seguir os passos:
3. Calcular o ganho de informação.
4. Selecionar o atributo com maior ganho de informação para o nó raiz da árvore.
5. Subdividir o conjunto T .
6. Repetir o procedimento para cada nó gerado.

A seguir, saiba mais sobre o classificador Naive Bayes.

Naive Bayes

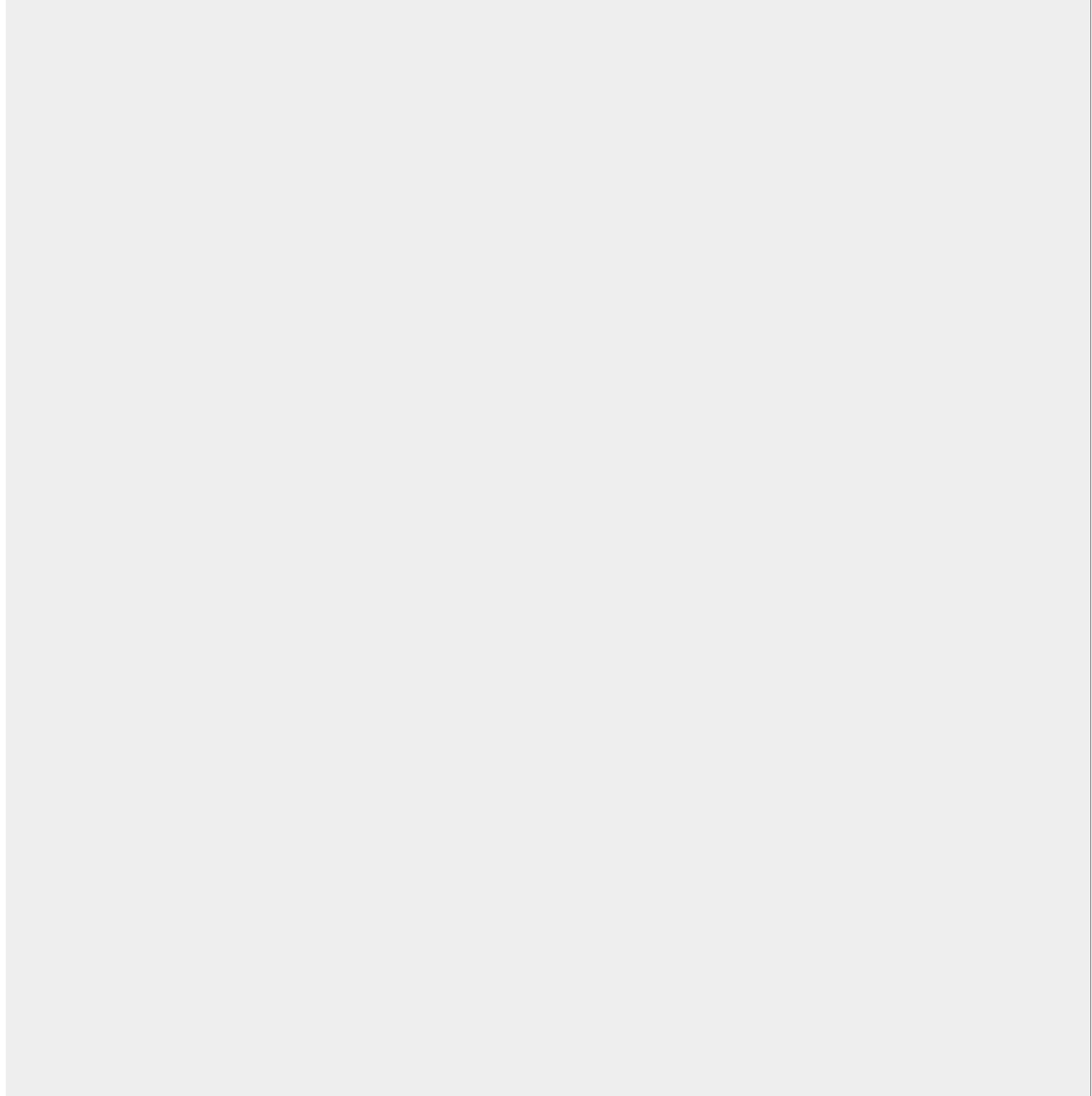
O Naive Bayes, ou Bayes Ingênuo, é um classificador genérico e de aprendizado dinâmico. É um dos métodos mais utilizados para classificação – especialmente em aplicações de *text mining*, previsões em tempo real e/ou sistemas embarcados –, por ser rápido computacionalmente e só necessitar de um pequeno número de dados de treinamento.

Ele é especialmente adequado quando o problema tem um grande número de atributos (características), e determina a probabilidade de um exemplo pertencer a uma determinada classe.



Esse método é chamado de ingênuo (*naive*, em inglês) porque **desconsidera completamente a correlação entre os atributos (características), tratando cada um de forma independente**. Além disso, o nome contém a palavra Bayes por ser baseado no teorema de Bayes, que determina a probabilidade de um evento com base em um conhecimento prévio (*a priori*) que pode estar relacionado a ele.

Entenda um pouco mais sobre o teorema de Bayes no infográfico a seguir:



O teorema de Bayes

Imagine que:

- X é uma observação (ou evidência) proveniente de um conjunto de n atributos;
- H é a hipótese de que a observação X pertença a uma determinada classe C .

Em problemas de classificação, queremos determinar $P(H|X)$, ou seja, a probabilidade de que a hipótese H se aplique a X .

Em outras palavras, buscamos a probabilidade de que a observação X pertença à classe C , uma vez que conhecemos a descrição de X .

Assim:

$P(H|X)$ é denominada a probabilidade *a posteriori* de H condicionada a X .

Por exemplo: suponha que tenhamos os dados de idade e renda mensal de clientes.

X é um cliente de 30 anos com renda mensal de R\$ 5.000,00, e H é a hipótese de esse cliente comprar um celular.

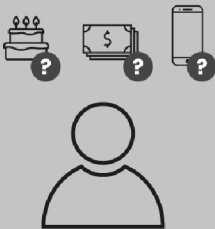
$P(H|X)$ é, então, a probabilidade de o cliente X comprar um celular, uma vez que conhecemos sua idade e renda mensal.



$P(H)$ é a probabilidade *a priori* de H .

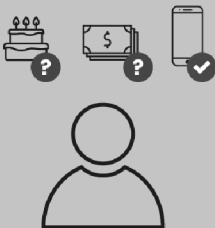
É a probabilidade de qualquer cliente, independentemente de idade, renda mensal ou qualquer outra informação, comprar um celular.

Essa probabilidade é baseada em mais informação do que $P(H)$, que é independente de X .



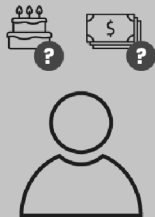
$P(X|H)$ é a probabilidade *a posteriori* de X condicionada a H

Por exemplo: a probabilidade de um cliente X ter 30 anos e ganhar R\$ 5.000,00, uma vez que comprou um celular.



$P(X)$ é a probabilidade *a priori* de X .

A probabilidade de que um cliente de nosso conjunto de dados tenha 30 anos e ganhe R\$ 5.000,00.



$P(H)$, $P(X|H)$ e $P(X)$ podem ser estimadas a partir do próprio conjunto de dados. O teorema de Bayes é útil para calcular a probabilidade *a posteriori* $P(H|X)$ a partir de $P(H)$, $P(X|H)$ e $P(X)$:

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}$$

Finalmente, podemos acompanhar a definição formal do algoritmo Naive Bayes:

Seja $X(A_1, A_2, ..., A_n, C)$ um conjunto de dados. Considere que $c_1, c_2, ..., c_k$ são as classes do problema (valores possíveis do atributo-alvo C) e que R é um registro que deve ser classificado. Sejam, ainda, $a_1, a_2, ..., a_n$ os valores que R assume para os atributos previsores $A_1, A_2, ..., A_n$,

respectivamente.

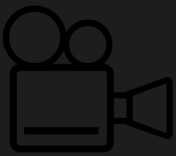
O algoritmo consiste em dois passos:

1. Calcular as probabilidades condicionais $P(C=ci|R)$, $i = 1, 2, ..., k$.
2. Indicar, como saída do algoritmo, a classe c , tal que $P(C=c|R)$ seja máxima, quando considerados todos os valores possíveis do atributo-alvo C .



A intuição por trás do algoritmo é dar maior peso para as classes mais frequentes e, conforme já discutimos, ele é chamado de ingênuo porque **considera, como hipótese, que os atributos são estatisticamente independentes entre si** – o que não ocorre em muitos casos práticos.

Entretanto, na prática, o método mostra-se muito efetivo, mesmo quando os previsores não são estatisticamente independentes.



Exemplo de Naive Bayes

Assista ao vídeo a seguir para ver um exemplo do funcionamento do Naive Bayes e entender melhor esse algoritmo.



A seguir, entenda mais sobre o algoritmo SVM (*support vector machine*).

SVM

O SVM (*support vector machine*, ou máquina de vetor de suporte) é um dos algoritmos mais efetivos para classificação. Pode ser aplicado em dados lineares ou não lineares.



Apesar de o treinamento dos modelos de SVM ser lento, eles exigem poucos ajustes, tendem a apresentar boa acurácia e conseguem modelar fronteiras de decisão complexas e não lineares. Além disso, são menos propensos a *overfitting*, se comparados com outros métodos.

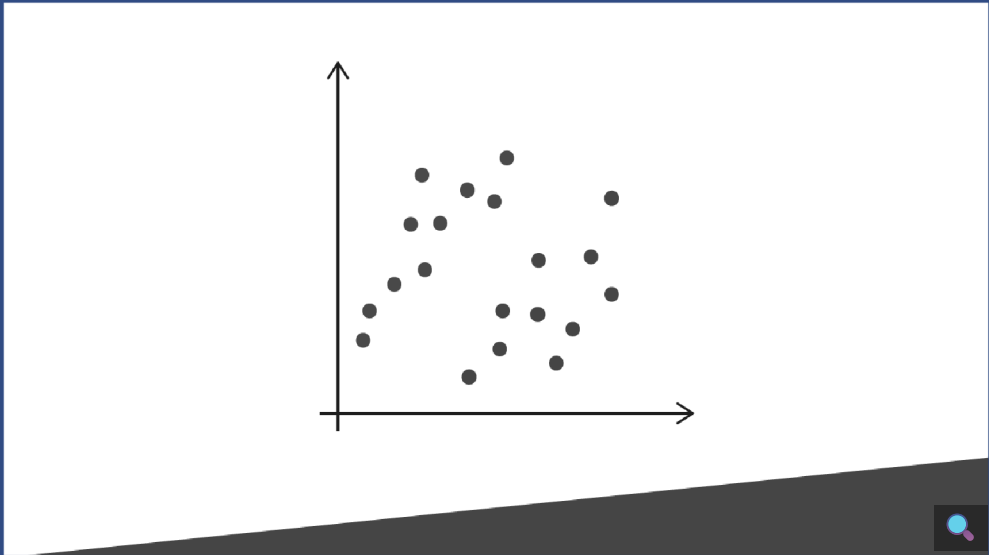
Essencialmente, o SVM realiza um mapeamento não linear para transformar os dados de treino originais em uma dimensão maior. Nessa nova dimensão, o algoritmo busca pelo hiperplano que separa os dados linearmente de forma ótima.

Com um mapeamento apropriado para uma dimensão suficientemente alta, dados de duas classes podem ser sempre separados por um hiperplano. O SVM encontra esse hiperplano usando vetores de suporte (exemplos essenciais para o treinamento) e margens definidas por eles.

Para melhor entendimento do SVM aplicado a problemas de classificação (uso mais comum), considere um conjunto de dados de treinamento linearmente separável na forma $\{x_i, y_i\}$, em que x_i corresponde ao vetor de dois atributos previsores e $y_i \in \{-1, 1\}$, às duas classes possíveis do problema. O conjunto de dados de entrada é utilizado para construir uma função de decisão $f(x)$, tal que:

- se $f(x) > 0$, então $y_i = 1$;
- se $f(x) < 0$, então $y_i = -1$.

A figura a seguir ilustra esse exemplo:



Conforme mencionado anteriormente, o algoritmo SVM constrói classificadores lineares que separam os conjuntos de dados por meio de um hiperplano. Considere hiperplano a generalização do conceito de plano para dimensões maiores que três.

Em um espaço p -dimensional, um hiperplano é um subespaço achatado de dimensão $p-1$ que não precisa passar pela origem.

Quando $d = 2$, o hiperplano é uma reta, e sua equação é dada por:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 = 0$$

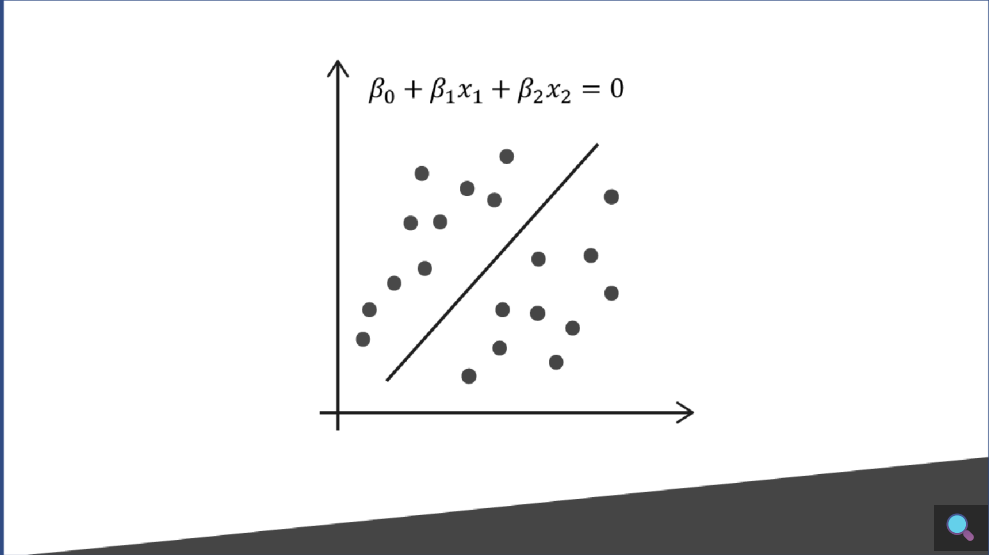
Aqui, β_1 e β_2 são parâmetros da reta que determinam a inclinação desta, e β_0 é o parâmetro que determina o ponto de corte do eixo y .

Nesse caso, o propósito do SVM é determinar parâmetros da reta (β_0 , β_1 e β_2) que permitam separar os conjuntos dos dados de treinamento em duas classes possíveis:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 < 0$$

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 > 0$$

Esse exemplo é ilustrado pela figura a seguir:



Da mesma forma que um plano tem duas dimensões e divide um conjunto tridimensional em dois espaços de dimensão três, um hiperplano em um espaço n -dimensional tem $n-1$ dimensões e o divide em dois subespaços de dimensão n .

Assim, para $d = p$, a equação do hiperplano é:

$$\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n = 0$$

Nesse caso, o hiperplano também divide o espaço p -dimensional em duas metades:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 > 0$$

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 < 0$$

Voltando ao exemplo anterior, como afirmamos previamente que o problema é linearmente separável, é possível construir um hiperplano que separe as observações de treino perfeitamente de acordo com seus rótulos de classe.

Esse hiperplano tem as seguintes propriedades:

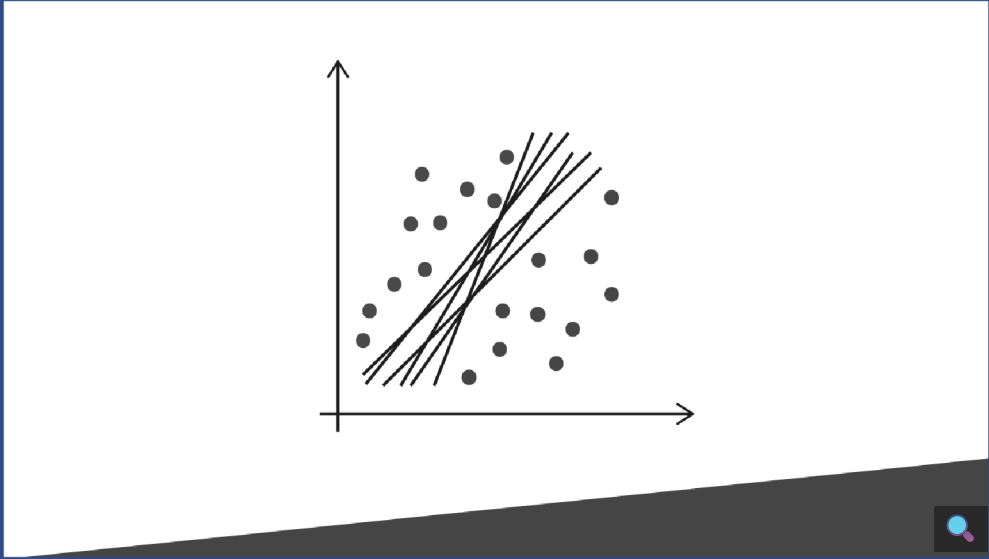
$$\beta_0 + \beta_1 x_i + \beta_2 x_2 > 0, \text{ se } y_i = 1$$

$$\beta_0 + \beta_1 x_i + \beta_2 x_2 < 0, \text{ se } y_i = -1$$

O hiperplano pode, então, ser usado para construir um classificador: o exemplo de teste recebe a classe, dependendo do lado do hiperplano

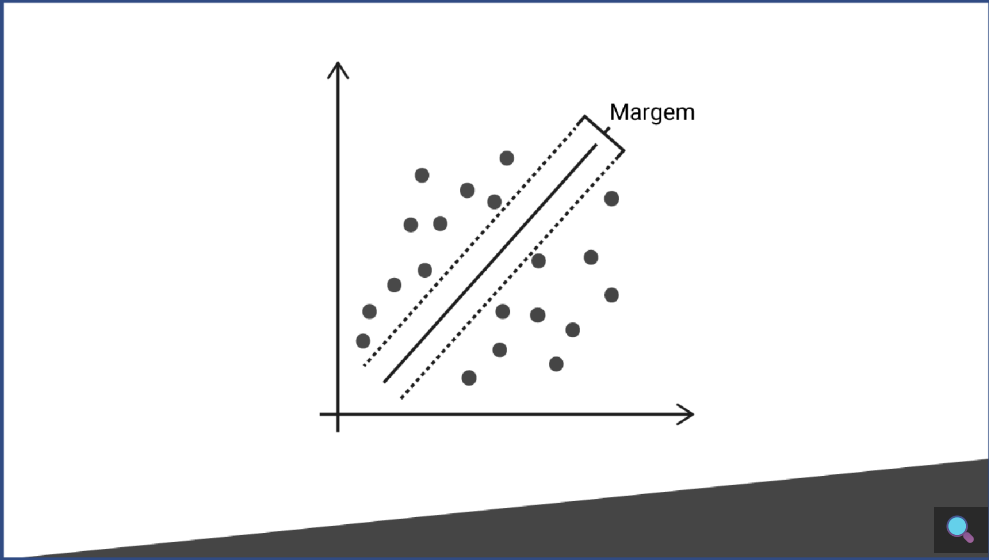
em que estiver localizado.

É importante notar que, no exemplo anterior, infinitas retas dividem corretamente o conjunto de treinamento em duas classes, conforme ilustrado na figura a seguir. O SVM deve, então, realizar um processo de escolha da reta separadora, entre o conjunto infinito de retas possíveis:

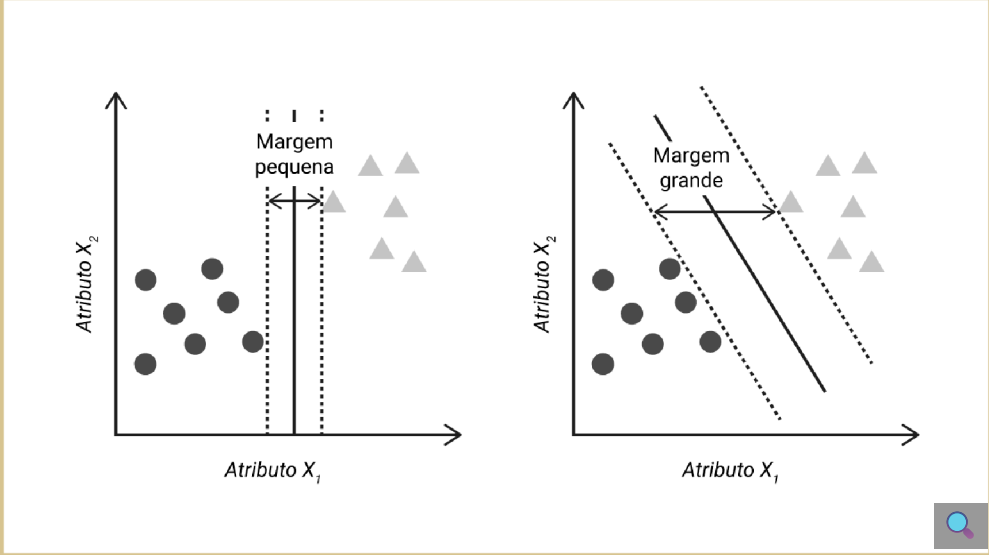


Na próxima figura, são apresentados apenas um classificador linear (ilustrado pela reta sólida) e duas retas pontilhadas paralelas a ele. Cada uma das retas pontilhadas é movida a partir da posição da reta sólida, determinando quando a reta paralela intercepta o primeiro ponto do conjunto de dados.

A margem é a distância construída entre essas duas retas paralelas pontilhadas:

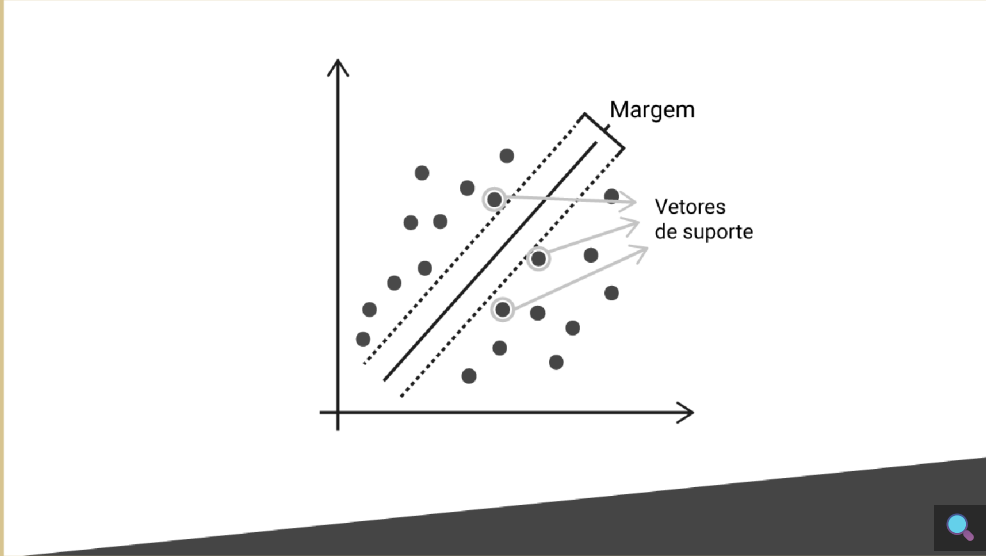


Assim como existem infinitas retas que separam os pontos em duas classes, há também diversos tamanhos de margem possíveis, dependendo da reta escolhida como classificador. A figura a seguir ilustra dois possíveis classificadores, com dois tamanhos de margem diferentes:



O classificador associado ao valor máximo de margem é denominado **classificador linear de margem máxima**. Os pontos do conjunto de dados de treinamento interceptados pelas linhas da margem são denominados vetores de suporte, e são os mais difíceis de classificar.

Os vetores de suporte são ilustrados pela figura a seguir:



Por construção, todos os vetores de suporte têm a mesma distância em relação à reta do classificador linear (a metade do comprimento da margem).



Mantendo o foco

É importante ressaltar que, apesar de o classificador linear de margem máxima ser geralmente bom, pode haver *overfitting* se o número de dimensões for grande.

O SVM realiza um processo de otimização, em que são determinados os parâmetros do classificador linear (β_0 , β_1 e β_2). O objetivo desse processo de otimização é determinar os valores de β_0 , β_1 e β_2 que produzem o valor máximo para o comprimento da margem.

A reta correspondente ao classificador linear é dita ótima porque, se for deslocada em alguma das duas direções das retas perpendiculares a ela, a probabilidade de haver um erro de classificação é maior. Assim, a posição do classificador linear correspondente ao comprimento de margem máximo é a mais segura possível com relação a eventuais erros de classificação e, quanto maior a distância de x para o hiperplano, maior a confiança sobre a classe a que x pertence.



A solução do problema de otimização do SVM pode ser obtida por meio de técnicas de programação quadrática, muito conhecidas na área de pesquisa operacional, que consistem em otimizar uma função quadrática sujeita a restrições lineares.

Uma vez obtidos os valores dos parâmetros β_0 , β_1 e β_2 , aplica-se uma função de decisão para classificar um novo exemplo x_i . A classe de x_i , y_i , é dada pelo sinal de $f(x)$:

$Se f(x_i) > 0, então y_i = 1$

$Se f(x_i) < 0, então y_i = -1$

Na prática, entretanto, os dados reais não costumam ser perfeitamente separáveis por um hiperplano. Além disso, o hiperplano margem máxima é extremamente sensível a mudanças em uma única observação, sugerindo que pode ocorrer *overfitting* nos dados de treino.

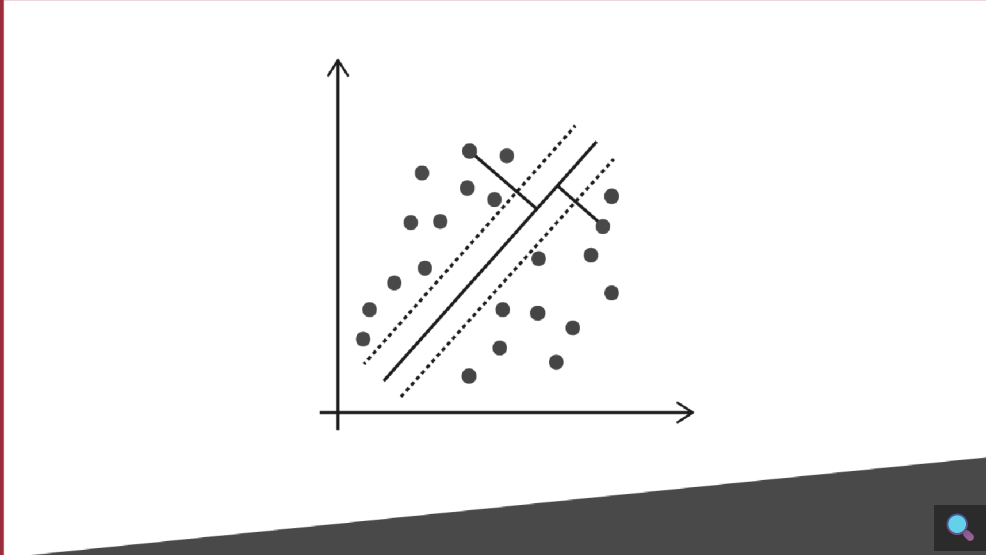
Por isso, podemos querer um classificador baseado em hiperplano que não separe perfeitamente as duas classes, com o objetivo de aumentar a robustez nas observações individuais e melhorar a classificação na maioria das observações de treino.

Ou seja, pode valer a pena classificar erroneamente algumas observações de treino a fim de melhorar a classificação nas observações restantes.

Interativo

Descrição do interativo

No exemplo ilustrado pela figura a seguir, o classificador cometeria erros na classificação para os dois pontos destacados, que podem ser considerados ruído:



Para isso, o parâmetro de custo C define a “rigidez” da margem e controla o *trade-off* entre o tamanho desta e o erro do classificador. Quanto maior for o valor de C , mais pontos podem ficar dentro da margem e maior será o erro de classificação, mas menor é a chance de *overfitting*. Se $C = 0$, a margem é rígida e temos o classificador de margem máxima. Na prática, esse classificador não produz bons resultados.

Observação: é importante ressaltar que na biblioteca scikit-learn o parâmetro C representa um parâmetro de regularização e funciona de forma contrária, adicionando uma penalidade a cada ponto classificado erradamente. Se o valor de C for pequeno, então, essencialmente, a penalidade para pontos classificados erradamente também será pequena, resultando em uma margem maior (mais flexível).

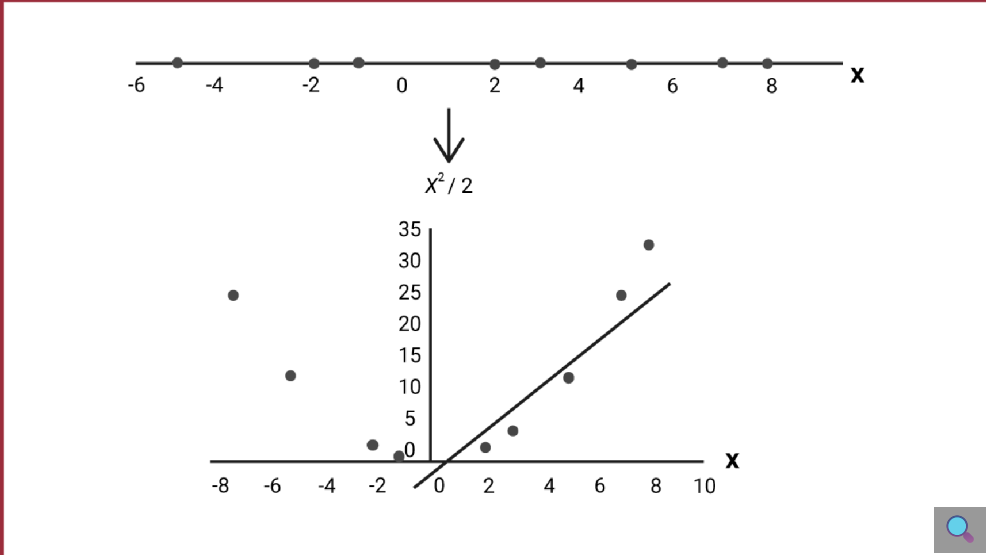
O número total de vetores de suporte depende da quantidade de folga permitida nas margens e da distribuição dos dados.

Quanto maior a folga permitida, maior o número de vetores de suporte e mais lenta a classificação dos dados de teste, pois a complexidade computacional do SVM está relacionada com o número de vetores de suporte.

Na prática, o SVM é implementado usando funções *kernel*, que, de forma simplificada, são objetos matemáticos que permitem que trabalhem em um espaço de dimensão maior. Nesse caso, em vez de se utilizarem as observações em si, é utilizado seu produto interno. A previsão de um novo exemplo é feita calculando o produto escalar entre o exemplo (x) e cada vetor de suporte (x_i). Os tipos de *kernel* mais utilizados são linear, polinomial e radial.

Quando há um conjunto de dados não linearmente separável, o SVM utiliza funções *kernel* para mapeá-lo para um espaço com dimensão maior. O classificador é, então, ajustado nesse novo espaço. O SVM é, na verdade, a combinação do classificador linear com um *kernel* não linear.

Esse processo é ilustrado pela figura a seguir:



O SVM também pode ser aplicado a problemas de classificação que envolvem múltiplas classes. Nesse caso, usa-se o algoritmo para treinar um modelo de classificação que informe se o registro é da classe c_i (região positiva) ou de alguma outra classe diferente (região negativa).

Dessa forma, constroem-se $p-1$ modelos de classificação, em que p é o número de classes possíveis no problema. Para classificar um novo exemplo, pode-se submetê-lo a cada um dos modelos de classificação gerados, e a classe selecionada para o exemplo é a

mais frequentemente atribuída.

Acompanhe agora um exemplo de classificação em Python.

Prática de *machine learning* em Python (classificação)

Para ilustrar os algoritmos de *machine learning* de classificação na prática, vamos examinar o popular *dataset* Wine, extraído de uma análise química de vinhos cultivados em uma mesma região da Itália, mas derivados de três produtores diferentes.



O objetivo desse *dataset* é identificar o produtor com base em 13 características químicas do vinho, ou seja, é um problema de classificação. O *dataset* contém 178 instâncias (linhas), sendo 59 do produtor 1, 71 do produtor 2 e 48 do produtor 3.

Vejamos um exemplo de código em Python usando a biblioteca scikit-learn, uma das mais utilizadas para *machine learning* com Python.

Interativo

Descrição do interativo

Iniciaremos esta prática importando os pacotes necessários para o *notebook*:

```
1 # Configuração para não exibir os warnings
2 import warnings
3 warnings.filterwarnings("ignore")
4
5 # Imports necessários
6 import pandas as pd
7 import numpy as np
8 import matplotlib.pyplot as plt
9 from sklearn.datasets import load_wine # para importar o dataset wine
10 from sklearn.model_selection import train_test_split # para particionar em bases de treino e teste (holdout)
11 from sklearn.model_selection import KFold # para preparar os folds da validação cruzada
12 from sklearn.metrics import accuracy_score # para executar a validação cruzada
13 from sklearn.neighbors import KNeighborsClassifier # para a exibição da acurácia do modelo
14 from sklearn.tree import DecisionTreeClassifier # algoritmo Árvore de Classificação
15 from sklearn.naive_bayes import GaussianNB # algoritmo Naive Bayes
16 from sklearn.svm import SVC # algoritmo SVM
```

A seguir, iremos preparar os dados. Para isso, carregaremos o *dataset* a partir da biblioteca scikit-learn, aplicaremos o *holdout* para efetuar a divisão em bases de treino (80%) e teste (20%) e separaremos em dez *folds* usando a validação cruzada:

```
1 # Carga do dataset
2 wine = load_wine()
3 dataset = pd.DataFrame(wine.data, columns=wine.feature_names) # conversão para dataframe
4 dataset['target'] = wine.target # adição da coluna target
5
6 dataset.head()
```

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	flavonoids	nonflavonoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline	target
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0	0
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0	0
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0	0
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0	1
4	13.24	2.59	2.87	21.0	118.0	2.80	2.99	0.39	1.82	4.32	1.04	2.93	735.0	1

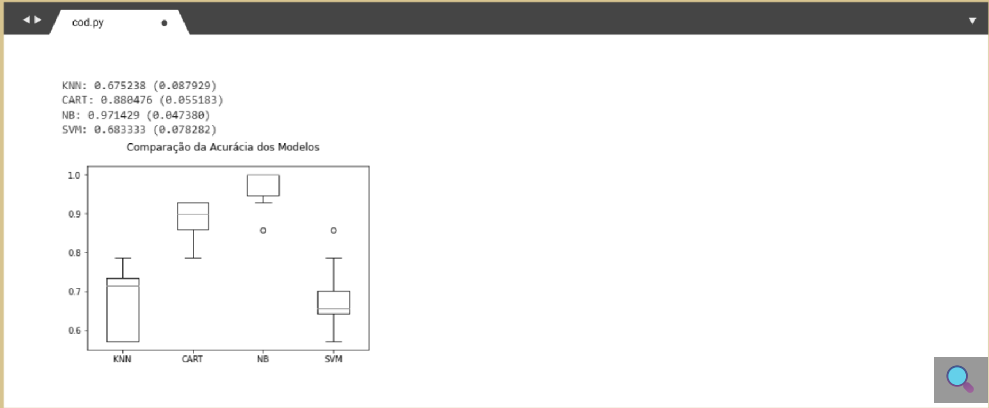
```
1 # Preparação dos dados
2
3 # Separação em bases de treino e teste (holdout)
4 array = dataset.values
5 X = array[:,0:10] # atributos
6 y = array[:,10] # classe (target)
7
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=7) # faz a divisão
9
10 # Criando os folds para a validação cruzada
11 num_particoes = 10 # número de folds da validação cruzada
12 kfold = KFold(n_splits=num_particoes, shuffle=True, random_state=7) # faz o particionamento em 10 folds
```

Em seguida, passaremos para a etapa de modelagem. Definiremos uma semente global para essa célula de código, necessária para garantir a reprodutibilidade do código com os mesmos resultados. Então, criaremos os modelos com os algoritmos KNN, Árvore de Classificação, Naive Bayes e SVM, adicionando-os em uma lista.

Depois, cada um desses modelos será treinado e avaliado com a base de treino, usando a validação cruzada 10-*fold*. O resultado médio da acurácia de cada modelo será impresso, bem como um gráfico *boxplot* resumizando os resultados das dez execuções (correspondentes aos dez *folds*):

```
1 # Modelagem
2
3 # Definindo uma seed global para esta célula de código
4 np.random.seed(7)
5
6 # Listas para armazenar os modelos, os resultados e os nomes dos modelos
7 models = []
8 results = []
9 names = []
10
11 # Preparando os modelos e adicionando-os em uma lista
12 models.append(('KNN', KNeighborsClassifier()))
13 models.append(('CART', DecisionTreeClassifier()))
14 models.append(('NB', GaussianNB()))
15 models.append(('SVM', SVC()))
16
17 # Avaliando um modelo por vez
18 for name, model in models:
19     cv_results = cross_val_score(model, X_train, y_train, cv=kfold, scoring='accuracy')
20     results.append(cv_results)
21     names.append(name)
22     msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std()) # média e desvio padrão dos 10 resultados da validação cruzada
23     print(msg)
24
25 # Boxplot de comparação dos modelos
26 fig = plt.figure()
27 fig.suptitle('Comparação da Acurácia dos Modelos')
28 ax = fig.add_subplot(111)
29 plt.boxplot(results)
30 ax.set_xticklabels(names)
31 plt.show()
```

A saída resultante será:



Analizando, verificamos que, considerando a acurácia média, o modelo treinado com o Naive Bayes apresentou os melhores resultados (97% de acurácia média e desvio-padrão de 5%), indicando que possivelmente seguiríamos com ele como escolha de algoritmo. Nesse caso, construiremos um novo modelo, treinado com toda a base de treino.

Esse modelo será avaliado utilizando a base de teste:

```
1 # Criando um modelo com todo o conjunto de treino
2 model = GaussianNB()
3 model.fit(X_train, y_train)
4
5 # Fazendo as predições com o conjunto de teste
6 predictions = model.predict(X_test)
7
8 # Estimando a acurácia no conjunto de teste
9 print(accuracy_score(y_test, predictions))
```

A saída resultante será:

1.0

Repare que conseguimos obter uma acurácia de teste de 100%.
Reforçamos que esse resultado não é comum quando trabalhamos com dados de problemas reais, mas é possível com *datasets* simplificados similares a esse, trabalhados e disponibilizados apenas para fins acadêmicos.

Ressaltamos que esse é um exemplo simples, apenas para ilustrar a aplicação de algoritmos de *machine learning* em problemas de classificação. Em problemas reais (com dados mais “sujos” e complexos), iríamos provavelmente trabalhar com algumas operações de pré-processamento de dados (como normalização e padronização para dados quantitativos, e *One-Hot-Encoding* para dados qualitativos nominais). Além disso, também experimentaríamos variar os hiperparâmetros dos algoritmos, o que resultaria em um código mais complexo, como veremos nas próximas aulas.



Reflita

Agora que você conheceu um pouco sobre problemas de classificação, chegou a hora de exercitar o cérebro e desenvolver ainda mais seu olhar crítico para uma situação do cotidiano. Com base no que estudamos nesta aula, reflita: quais problemas reais, encontrados nas empresas, poderiam ser resolvidos com o apoio de modelos de *machine learning* de classificação?



Referências

Clique aqui para acessar as referências e créditos desta aula.



BISHOP, C. M.; NASRABADI, N. M. **Pattern recognition and machine learning**. New York: Springer, 2006.

BURKOV, A. **The hundred-page machine learning book**. Québec City: Andriy Burkov, 2019.

ESCOVEDO, T.; KOSHIYAMA, A. **Introdução a data science**: algoritmos de *machine learning* e métodos de análise. São Paulo: Casa do Código, 2020.

GÉRON, A. **Mãos à obra**: aprendizado de máquina com scikit-learn, Keras & TensorFlow – conceitos, ferramentas e técnicas para a construção de sistemas inteligentes. Sebastopol: O’Reilly Media, 2021.

KALINOWSKI, M.; ESCOVADO, T.; VILLAMIZAR, H.; LOPES, H. **Engenharia de software para ciência de dados**: um guia de boas práticas com ênfase na construção de sistemas de machine learning em python. São Paulo: Casa do Código, 2023.

MURPHY, K. P. **Machine learning**: a probabilistic perspective. Cambridge: MIT Press, 2012.

VANDERPLAS, J. **Python data science handbook**: essential tools for working with data. Sebastopol: O’Reilly Media, 2016.

Bancos de imagens Pexels, Pixabay, Envato, Freepik e Shutterstock